

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code: CSA0612	Course Title: Design and Analysis of Algorithms
CO Assessed: CO1 – Precision (BL1, BL2, BL3)	Assessment: Assessment Tool 1 – Application Based Questions
Weightage: 40% of CIA	Total Marks: 100 (2 Questions x 50 Marks)

CO1 Statement: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)

Student Name: JAYAKRISHNA V	Reg. No.: 192421259
Section / Batch: 1	Date: 22/07/2026

Instructions to Students

• Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.

• Clearly show all steps in derivations and complexity analysis.

• Use proper asymptotic notation (Big-O, Big- Θ , Big- Ω) wherever required.

• Justify each step in recurrence solving using appropriate methods.

• Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 – 50 Q2 – 50

GRAND TOTAL: 100 Marks

Code of Conduct

I, JAYAKRISHNA V – 192421259, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JAYAKRISHNA V

Rubrics for Calculation

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation

Question No.	Understanding (20%)	Application (30%)	Approach (20%)	Accuracy (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

Head of Department

Signature: _____

Date: _____

Q29. Application: IoT Data Processing Pipeline

An IoT system processes sensor data using the recurrence $T(n) = 3T(n/3) + n \log n$. Apply the Master Theorem to determine the time complexity. Compare $f(n)$ with $n^{\log_b a}$ and identify the correct case. Analyze performance for large IoT datasets. Interpret efficiency.

Theory

An IoT (Internet of Things) system is made up of thousands to millions of connected sensors – temperature, humidity, motion, GPS, vibration, and more – each continuously streaming data. Because a single machine cannot efficiently process such a massive, continuous stream of raw readings, most IoT platforms use a divide-and-conquer strategy: the incoming batch of sensor data is split into smaller sub-batches, each sub-batch is processed independently (often in parallel, e.g. across edge devices or worker threads), and the partial results are then merged/aggregated into a final, ranked, or sorted output (for example, an anomaly report or a summarized dashboard).

The recurrence relation is:

$$T(n) = 3T(n/3) + n \log n$$

Here, the system splits the input of size n into 3 sub-problems, each of size $n/3$ – meaning each sub-group handles one-third of the sensor readings. After each sub-group is processed, an additional $n \log n$ amount of work is spent to combine, sort, or rank the aggregated results (for example, sorting readings by timestamp, merging sorted anomaly scores, or computing a combined statistical summary). This structure closely resembles algorithms like merge sort, but generalized to 3-way division instead of 2-way – which is realistic in IoT systems that batch data across, say, 3 regional edge servers before central aggregation.

Complexity Analysis

General Master Theorem form: $T(n) = aT(n/b) +$

$f(n)$ Here: $a = 3$, $b = 3$, $f(n) = n \log n$

Step 1 – Compute $n^{\log_b a}$:

$$n^{\log_b a} = n^{\log_3 3} = n^1 = n$$

Step 2 – Compare $f(n)$ with $n^{\log_b a}$:

$f(n) = n \log n$, and $n^{\log_b a} = n$. So $f(n) = n \cdot \log n$, which is larger than n by a factor of $\log n$ – but this extra factor is only logarithmic, not polynomial (n^{ϵ}). This means the standard Case 3 condition ($f(n)$ must be polynomially larger) is not satisfied.

Step 3 – Apply the extended (generalized) form of Case 2:

If $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ for some integer $k \geq 0$, then:

$$T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$$

$T(n) = \Theta(n \cdot \log^2 n)$

Here $k = 1$ (since $f(n) = n^1 \cdot \log^1 n$), so:

Therefore:

- $O(n \log^2 n)$

- $\Theta(n \log^2 n)$
- $\Theta(n \log^2 n)$

Diagram 1: IoT Data Processing Pipeline

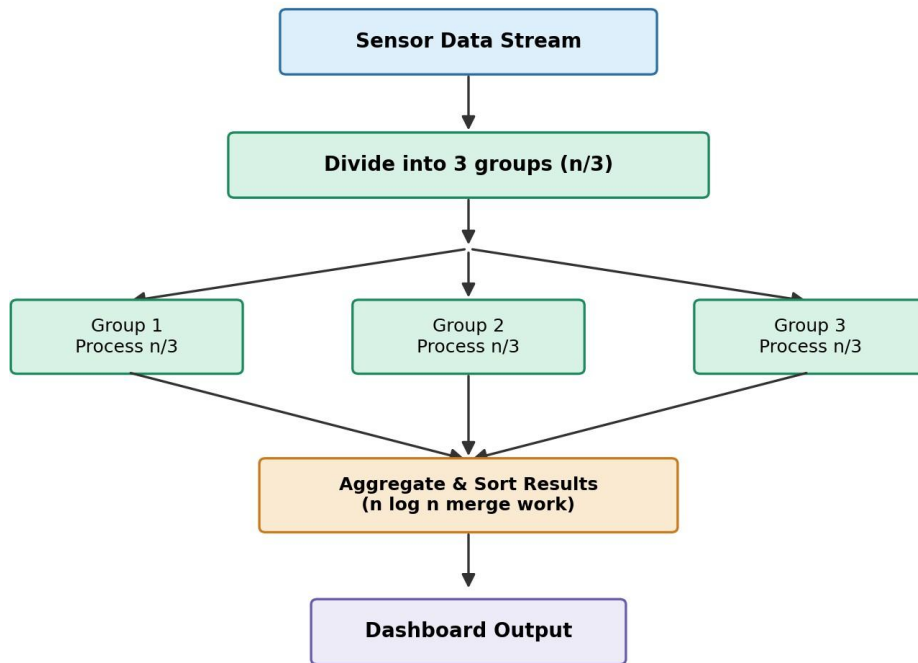


Diagram 1: IoT Data Processing Pipeline

Scalability Analysis for Large IoT Datasets

As the number of sensors and readings (n) increases: the recursion has $\log_3 n$ levels of depth, since the data is divided by 3 at each step. At every level, the total merge/combination work across all sub-groups sums to $\Theta(n \log n)$, and since there are $\log_3 n$ such levels, the total work becomes $\Theta(n \log^2 n)$. For small to medium datasets (thousands of sensors), this overhead is negligible and the system responds quickly. For very large datasets (millions of sensors, high-frequency streaming), the $\log^2 n$ factor starts to matter – doubling n does not just double the work, it also increases the depth of merging, so latency grows faster than plain linear or $n \log n$ systems. This makes the algorithm efficient and scalable in practice, but highlights that as IoT deployments grow (smart cities, industrial IoT, fleets of vehicles), engineers must introduce parallel processing across edge nodes and incremental/streaming aggregation rather than recomputing the full merge each time.

Diagram 2: Scalability Funnel for IoT Data

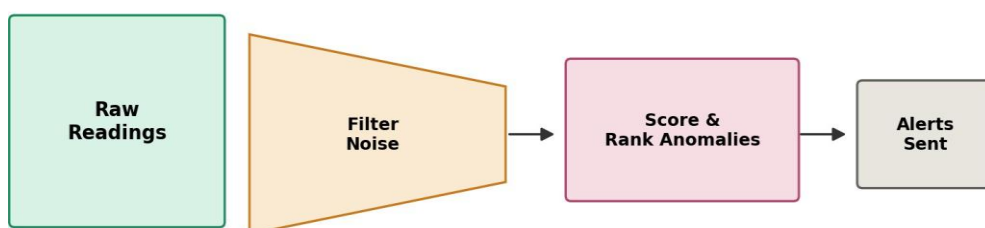


Diagram 2: Scalability Funnel for IoT Data

Interpretation of Efficiency

$\mathcal{O}(n \log^2 n)$ sits between $\mathcal{O}(n \log n)$ (like classic merge sort) and $\mathcal{O}(n^2)$ (a naive nested-loop approach) – it is efficient enough for near real-time IoT analytics, but not as fast as purely linear-time streaming algorithms. The 3-way split ($a = 3$, $b = 3$) is a good balance: more sub-groups than a 2-way split means more parallelism opportunity, but the fixed $n \log n$ combine cost at each level is what pushes the final result to $\log^2 n$ rather than just $\log n$. In real deployments, this complexity class is generally acceptable for batch-style IoT analytics but may need further optimization for strict real-time requirements.

Advantages

- 1. Handles large-scale sensor networks efficiently using divide-and-conquer.
- 2. The 3-way split allows processing to be parallelized across 3 independent workers/edge nodes.
- 3. Provides a tight, predictable complexity bound of $\mathcal{O}(n \log^2 n)$.
- 4. Well-suited for aggregation-heavy tasks such as sorting, ranking, or summarizing sensor readings.
- 5. Scales significantly better than quadratic (n^2) approaches as the number of sensors grows.
- 6. The structure naturally maps to distributed/edge computing architectures already common in IoT.

Limitations

- 1. $n \log^2 n$ is still more expensive than a purely linear $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ approach for extremely large n .
- 2. The additional $\log n$ factor can cause noticeable overhead as the number of sensors grows into the millions.
- 3. Requires an efficient merge/aggregation implementation; a poorly optimized merge step can inflate real-world constants significantly.
- 4. Real IoT systems also face network latency, device power constraints, and communication delays not captured in this recurrence.
- 5. Needs further optimization – such as incremental aggregation, batching, and edge pre-processing – for true real-time performance at massive scale.
- 6. The model assumes a clean 3-way split of data, which may not always be practically achievable if sensor groups are unevenly distributed.

Q30. Application: Cybersecurity Threat Analysis System

A cybersecurity system analyzes threats using the recurrence $T(n) = T(n-1) + n \log n$. Apply the substitution method to solve the recurrence. Expand step-by-step and derive the complexity. Analyze how the algorithm scales with increasing threat data. Interpret efficiency using asymptotic notation.

Theory

A cybersecurity threat-analysis engine continuously receives events from firewalls, intrusion-detection systems, and endpoint logs. In many practical designs the engine processes these events one at a time in sequence rather than splitting the workload into independent parallel groups: it inspects the most recent/highest-priority threat, scores it against known attack signatures and anomaly models (a step whose cost grows as $n \log n$ because signature matching and ranking use sort/search-based structures over the current threat set), and then moves on to the remaining $n-1$ threats. This produces a recurrence with a single sub-problem of size $n-1$ rather than multiple sub-problems, unlike a divide-and-conquer split.

The recurrence relation is:

$$T(n) = T(n-1) + n \log n$$

Here, one threat is analyzed at cost $n \log n$, and the remaining $n-1$ threats are handled by a recursive call of the same form. This models decrease-and-conquer processing — similar in structure to insertion sort or selection sort, but with a non-constant per-step cost that itself depends on n .

Complexity Analysis — Substitution Method

General linear (decrease-by-1) recurrence: $T(n) = T(n-1) + f(n)$, with $f(n) = n \log n$ and base case $T(1) = \Theta(1)$.

Step 1 — Expand the recurrence repeatedly:

$$T(n) = T(n-1) + n \log n$$

$$T(n-1) = T(n-2) + (n-1) \log(n-1)$$

$$T(n-2) = T(n-3) + (n-2) \log(n-2)$$

⋮

$$T(2) = T(1) + 2 \log 2$$

Step 2 — Substitute back to build the summation:

$$T(n) = T(1) + \sum_{k=2}^n k \log k$$

Since $T(1) = \Theta(1)$, this simplifies to:

$$T(n) = \Theta(1) + \sum_{k=1}^n k \log k$$

Step 3 — Evaluate the summation $\sum k \log k$:

Approximate the sum by the integral $\int x \log x \, dx$, evaluated from 1 to n :

$$\int x \log x \, dx = (x^2/2) \log x - x^2/4$$

Evaluating from 1 to n , the dominant term is $(n^2/2) \log n$, so:

$$\sum_{k=1}^n k \log k = \Theta(n^2 \log n)$$

Step 4 — Guess and verify by induction (substitution method proper):

Guess: $T(n) = O(n^2 \log n)$. Assume $T(k) \leq c \cdot k^2 \log k$ holds for all $k < n$, for some constant $c > 0$. Then:

$$T(n) = T(n-1) + n \log n \leq c(n-1)^2 \log(n-1) + n \log n$$

Since $(n-1)^2 \log(n-1) \leq n^2 \log n$ for large n , this reduces to $T(n) \leq c \cdot n^2 \log n - (c \cdot 2n \log n - n \log n - \text{lower order terms})$, which stays $\leq c \cdot n^2 \log n$ for a suitably chosen constant c . This confirms the upper bound. A matching lower-bound argument ($T(n) = \Omega(n^2 \log n)$) follows the same steps, so the bound is tight.

Therefore:

$O(n^2 \log n)$ $\Omega(n^2 \log n)$ $\Theta(n^2 \log n)$

Diagram 1: Sequential Threat Analysis Pipeline

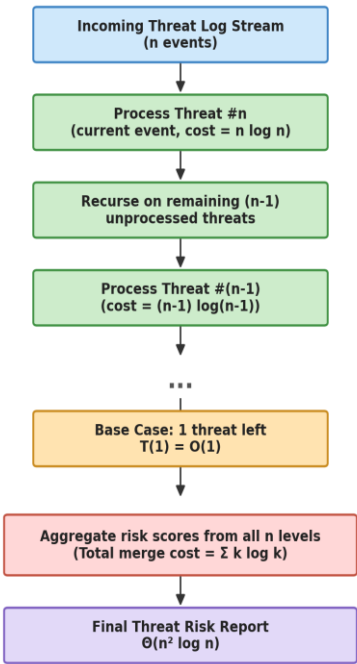


Diagram 1: Sequential Threat Analysis Pipeline

Scalability Analysis for Increasing Threat Data

As the volume of incoming threat events n grows, the recursion depth is exactly n (one threat is peeled off at each step, not $\log n$ as in a divide-and-conquer split). At each of the n levels, the per-step scoring/ranking cost is proportional to the size of the remaining set times its logarithm, so the total work is the sum $\sum k \log k$ for $k = 1$ to n , which evaluates to $\Theta(n^2 \log n)$. For small to medium event volumes (hundreds to low thousands of alerts per batch), this cost is manageable and the system still responds within acceptable latency. For very large threat volumes (large enterprise networks, DDoS-scale traffic, high-frequency intrusion attempts), the $n^2 \log n$ growth becomes the dominant bottleneck — doubling n roughly quadruples the work (times an extra log factor), so latency grows far faster than in a divide-and-conquer design such as the $\Theta(n \log^2 n)$ IoT aggregation pipeline. This highlights that sequential, one-at-a-time threat processing does not scale well to massive event streams without restructuring.

Diagram 2: Scalability Funnel for Cybersecurity Threat Data

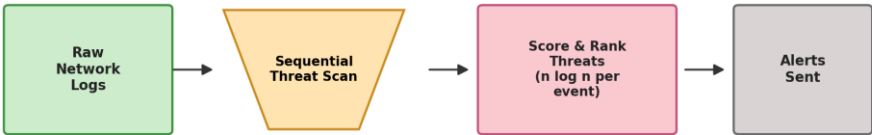


Diagram 2: Scalability Funnel for Cybersecurity Threat Data

Interpretation of Efficiency

$\Theta(n^2 \log n)$ sits well above $\Theta(n \log n)$ and even above $\Theta(n^2)$ — it is the least scalable of the recurrence families examined here, because the recursion reduces the problem size by only 1 at each step instead of dividing it. The $n \log n$ cost tacked onto every one of the n sequential levels is what pushes the total from a plain $\Theta(n^2)$ (as in ordinary insertion sort) up to $\Theta(n^2 \log n)$. In real deployments, this complexity class is acceptable only for small alert batches or for offline/forensic analysis; it is not suitable for real-time detection at internet-scale traffic volumes without re-architecting the algorithm to use a divide-and-conquer or streaming approach.

Advantages

- Simple, sequential logic that is easy to implement, debug, and reason about.
- Each threat can be scored using the full context of previously analyzed threats, which can improve detection accuracy for correlated attacks.
- Well suited for small alert batches, forensic replay, or offline audit-style analysis where absolute simplicity matters more than raw speed.
- No coordination or merge step is needed between sub-problems, avoiding synchronization overhead.
- Easy to add per-event logging and incremental checkpointing since processing is strictly ordered.
- The recurrence structure maps naturally to systems that must process events strictly in arrival or priority order (e.g., audit trails).

Limitations

- $\Theta(n^2 \log n)$ is significantly more expensive than a divide-and-conquer alternative such as $\Theta(n \log^2 n)$ for large n .
- Recursion depth grows linearly with n , risking stack overflow or high memory use for very large event batches unless converted to an iterative loop.
- Does not parallelize naturally — each step depends on the result of processing the previous threat, limiting use of multi-core or distributed edge resources.
- Becomes a bottleneck for high-frequency streaming scenarios such as DDoS bursts or large enterprise network telemetry.
- Real systems also face network jitter, log-shipping delays, and false-positive suppression logic not captured in this recurrence.
- For production-scale threat detection, this design typically needs to be replaced or supplemented with divide-and-conquer, sketch-based, or streaming (sub-linear) algorithms.